

Memory Leaks in React Forms:

Rohit is sitting in an interview.

Interviewer:

Rohit, how does memory leakage happen in different types of React forms?

How Rohit Inspects Memory Leaks

First, I try to repeat the problem by opening and closing the form many times.

Then, I use **Chrome DevTools** → **Memory tab** → **Heap Snapshots**.

- Take a snapshot before opening the form
- Open and close form multiple times
- Take another snapshot

If memory keeps growing, it means there is a leak.

How does Rohit handle memory leaks that happen in different types of React forms?

1. Controlled Form

React 19 encourages better state handling and concurrency, but controlled forms still work similarly.

Example:

```
import React, { useState, useEffect } from "react";
```

```
function ControlledForm() {  
  const [name, setName] = useState("");  
  
  useEffect(() => {  
    const id = setInterval(() => {  
      console.log("Controlled form running");  
    }, 1000);  
  
    return () => clearInterval(id); // Cleanup to stop leak  
  }, []);  
  
  return (  
    <input  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
      placeholder="Controlled Input"  
    />  
  );  
}
```

Pros: Full control, validation easy

Cons: Too many re-renders → can cause lag

Risk of memory leak: If event listeners or timers are not cleaned

2. Uncontrolled Form:

Example:

```
import React, { useRef, useEffect } from "react";
```

```
function UncontrolledForm() {
```

```
  const inputRef = useRef();
```

```
  useEffect(() => {
```

```
    function onFocus() {
```

```
      console.log("Input focused");
```

```
    }
```

```
    const input = inputRef.current;
```

```
    input.addEventListener("focus", onFocus);
```

```
    return () => input.removeEventListener("focus", onFocus); //  
Cleanup
```

```
  }, []);
```

```
return <input ref={inputRef} placeholder="Uncontrolled Input" />
}
```

Pros: Fast, fewer re-renders

Cons: No instant React validation

Risk of memory leak: Rare, simpler cleanup

3. React Hook Form:

Example:

```
import React, { useEffect } from "react";
import { useForm } from "react-hook-form";

function ReactHookForm() {
  const { register, watch } = useForm();

  useEffect(() => {
    const subscription = watch((value) => {
      console.log("React Hook Form value:", value);
    });

    return () => subscription.unsubscribe(); // Cleanup
  }, [watch]);

  return (
    <form>
```

```
    <input {...register("name")} placeholder="React Hook Form" />
  </form>
);
}
```

Pros: Very fast, no unnecessary re-renders

Cons: Learning curve

Risk of memory leak: Very low

4. Formik:

Example:

```
import React, { useEffect } from "react";
import { Formik, Form, Field } from "formik";

function FormikForm() {
  useEffect(() => {
    const id = setInterval(() => {
      console.log("Formik running");
    }, 2000);

    return () => clearInterval(id); // Cleanup
  }, []);
}
```

```
return (  
  <Formik  
    initialValues={{ name: "" }}  
    onSubmit={(values) => console.log(values)}  
  >  
    <Form>  
      <Field name="name" placeholder="Formik Input" />  
      <button type="submit">Submit</button>  
    </Form>  
  </Formik>  
}
```

Pros: Very powerful for big forms

Cons: Slower, more re-renders

Risk of memory leak: If many fields + heavy validation